LATERAL Joins in PostgreSQL

Table of Contents

1. Learning Objectives
2. Theory: What is LATERAL?
3. LATERAL with Subqueries
4. LATERAL with Set-Returning Functions
5. LATERAL vs Correlated Subquery
6. Top-N per Group with LATERAL
7. LATERAL with LIMIT
8. LATERAL for Complex Calculations
9. LEFT JOIN LATERAL
10. ASCII Visual Diagrams
11. Common Mistakes
12. Best Practices
13. Performance Considerations
14. Interview Questions & Answers
15. Hands-on Exercises
16. Advanced Notes
17. Cross-references

Learning Objectives

By the end of this module, you will be able to:

- Understand when and why to use LATERAL joins
- Use LATERAL to solve top-N per group problems
- Apply LATERAL with set-returning functions
- Distinguish LATERAL from correlated subqueries
- Optimize queries using LATERAL instead of complex subqueries

---

Theory: What is LATERAL?

In a normal FROM clause, each item is independent — a subquery cannot reference columns from preceding tables. The LATERAL keyword removes this restriction, allowing a subquery in FROM to reference columns from tables that appear before it in the FROM list.

```
Normal FROM (no cross-reference):
FROM table_a, (SELECT * FROM table_b) sub   ← sub cannot reference table_a

LATERAL FROM (allows cross-reference):
FROM table_a, LATERAL (SELECT * FROM table_b WHERE b.col = table_a.col) sub
                                                              ↑
                                                    references table_a!
```

LATERAL effectively creates a correlated subquery in the FROM clause, executed once per row of the preceding table — similar to a for-each loop.

LATERAL with Subqueries

Syntax

```
SELECT *
FROM table_a,
     LATERAL (subquery referencing table_a) AS alias;

-- or with explicit JOIN:
SELECT *
FROM table_a
JOIN LATERAL (subquery referencing table_a) AS alias ON TRUE;

-- or LEFT JOIN for outer behavior:
SELECT *
FROM table_a
LEFT JOIN LATERAL (subquery referencing table_a) AS alias ON TRUE;
```

Examples

```
-- Example 1: Most recent order for each customer
SELECT
    c.customer_id,
    c.customer_name,
    latest_order.order_id,
    latest_order.order_date,
    latest_order.total_amount
FROM customers c
LEFT JOIN LATERAL (
    SELECT order_id, order_date, total_amount
    FROM orders o
    WHERE o.customer_id = c.customer_id
    ORDER BY order_date DESC
    LIMIT 1
) AS latest_order ON TRUE
ORDER BY c.customer_name;

-- Example 2: Top-2 orders per customer
SELECT
    c.customer_name,
    top_orders.order_id,
    top_orders.order_date,
    top_orders.total_amount,
    top_orders.rank
FROM customers c
LEFT JOIN LATERAL (
    SELECT
        order_id,
        order_date,
```

```
        total_amount,
        ROW_NUMBER() OVER (ORDER BY total_amount DESC) AS rank
    FROM orders o
    WHERE o.customer_id = c.customer_id
    ORDER BY total_amount DESC
    LIMIT 2
) AS top_orders ON TRUE
ORDER BY c.customer_name, top_orders.rank;

-- Example 3: LATERAL for computed columns that depend on other columns
SELECT
    e.emp_name,
    e.salary,
    e.department,
    dept_stats.avg_sal,
    dept_stats.max_sal,
    e.salary - dept_stats.avg_sal AS vs_avg
FROM emp_salary e
JOIN LATERAL (
    SELECT
        AVG(salary) AS avg_sal,
        MAX(salary) AS max_sal
    FROM emp_salary
    WHERE department = e.department
) AS dept_stats ON TRUE
ORDER BY e.department, e.salary DESC;

-- Example 4: LATERAL subquery with multiple result columns
SELECT
    m.month_date,
    m.department,
    m.revenue,
    prev.revenue        AS prev_month_rev,
    prev.month_date     AS prev_month_date,
    m.revenue - COALESCE(prev.revenue, 0) AS change
FROM monthly_revenue m
LEFT JOIN LATERAL (
    SELECT revenue, month_date
    FROM monthly_revenue m2
    WHERE m2.department = m.department
      AND m2.month_date < m.month_date
    ORDER BY m2.month_date DESC
    LIMIT 1
) AS prev ON TRUE
ORDER BY m.department, m.month_date;

-- Example 5: LATERAL for percentile within context
SELECT
    e.emp_name,
    e.department,
    e.salary,
    dept_p.p50,
```

```
    dept_p.p90,
    CASE WHEN e.salary >= dept_p.p90 THEN 'Top 10%'
         WHEN e.salary >= dept_p.p50 THEN 'Above Median'
         ELSE 'Below Median'
    END AS salary_tier
FROM emp_salary e
JOIN LATERAL (
    SELECT
        PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary) AS p50,
        PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY salary) AS p90
    FROM emp_salary
    WHERE department = e.department
) AS dept_p ON TRUE
ORDER BY e.department, e.salary DESC;
```

LATERAL with Set-Returning Functions

Set-returning functions (SRFs) in PostgreSQL implicitly use LATERAL when placed in FROM. This is their normal usage pattern.

```
-- Example 6: UNNEST with LATERAL (implicit)
SELECT emp_name, skill
FROM emp_salary,
     UNNEST(ARRAY['SQL', 'Python', 'Git']) AS skill;
-- Each employee gets all 3 skills (cross product via unnest)

-- Example 7: generate_series with LATERAL
SELECT
    c.customer_name,
    gs.n AS month_offset,
    CURRENT_DATE + (gs.n || ' months')::INTERVAL AS target_date
FROM customers c,
     LATERAL generate_series(1, 3) AS gs(n)
ORDER BY c.customer_name, gs.n;

-- Example 8: JSON array expansion with LATERAL
SELECT
    j.data->>'name' AS name,
    elem
FROM (VALUES
    ('{"name":"Alice","tags":["SQL","Python"]}'::JSONB),
    ('{"name":"Bob","tags":["Java","Go"]}')
) AS j(data),
LATERAL jsonb_array_elements_text(j.data->'tags') AS elem;

-- Example 9: LATERAL for string splitting
SELECT
    rep_name,
    word
FROM sales,
```

```
     LATERAL STRING_TO_TABLE(product, ' ') AS word
WHERE region = 'North';
```

LATERAL vs Correlated Subquery

```
Correlated Subquery:          LATERAL Join:
──────────────────────        ──────────────────────────────────────

Can only return scalar        Can return multiple rows AND columns
values in SELECT              in the FROM clause

Cannot use LIMIT/ORDER BY     Can use LIMIT, ORDER BY, complex logic

Must be rewritten as          Natural expression in FROM
a scalar expression

Example: customer's           Top-3 orders per customer (impossible
last order date               with correlated subquery, easy with LATERAL)
```

```sql
-- CORRELATED SUBQUERY: one value only
SELECT
    c.customer_name,
    (SELECT MAX(order_date) FROM orders WHERE customer_id = c.customer_id) AS
last_order
FROM customers c;

-- LATERAL: multiple columns and rows
SELECT
    c.customer_name,
    lat.order_date,
    lat.total_amount
FROM customers c
LEFT JOIN LATERAL (
    SELECT order_date, total_amount
    FROM orders WHERE customer_id = c.customer_id
    ORDER BY order_date DESC LIMIT 1
) AS lat ON TRUE;
```

Top-N per Group with LATERAL

LATERAL is the cleanest solution for top-N per group (alternative to ROW_NUMBER subquery).

```sql
-- Example 10: Top-3 products by revenue per customer
-- (simulated with sales data)
SELECT
    r.region,
    top_reps.rep_name,
    top_reps.revenue,
    top_reps.rank
```

```
FROM (SELECT DISTINCT region FROM sales WHERE region IS NOT NULL) r
LEFT JOIN LATERAL (
    SELECT
        rep_name,
        SUM(amount) AS revenue,
        RANK() OVER (ORDER BY SUM(amount) DESC) AS rank
    FROM sales s
    WHERE s.region = r.region
    GROUP BY rep_name
    ORDER BY revenue DESC
    LIMIT 3
) AS top_reps ON TRUE
ORDER BY r.region, top_reps.rank;

-- Example 11: Latest N events per user (efficient with index)
SELECT
    c.customer_name,
    recent.order_id,
    recent.order_date,
    recent.total_amount
FROM customers c
LEFT JOIN LATERAL (
    SELECT order_id, order_date, total_amount
    FROM orders o
    WHERE o.customer_id = c.customer_id
    ORDER BY order_date DESC
    LIMIT 3
) AS recent ON TRUE
ORDER BY c.customer_name, recent.order_date DESC;
```

LATERAL with LIMIT

LATERAL + LIMIT is the canonical solution for "most recent N records per group" — a pattern that is expensive or impossible without LATERAL.

```
-- Example 12: Most recent 2 orders per customer
SELECT c.customer_name, lat.*
FROM customers c,
LATERAL (
    SELECT order_date, total_amount
    FROM orders
    WHERE customer_id = c.customer_id
    ORDER BY order_date DESC
    LIMIT 2
) AS lat;
-- Each customer row triggers exactly one index scan with LIMIT
-- Very efficient compared to ROW_NUMBER subquery for large datasets
```

LATERAL for Complex Calculations

```
-- Example 13: LATERAL for multi-step calculation
SELECT
    e.emp_name,
    e.salary,
    e.department,
    calc.years_employed,
    calc.annual_raise,
    calc.projected_salary_5yr
FROM emp_salary e
JOIN LATERAL (
    SELECT
        EXTRACT(YEAR FROM AGE(CURRENT_DATE, e.hire_date)) AS years_employed,
        e.salary * 0.05                                   AS annual_raise,
        e.salary * POWER(1.05, 5)                         AS projected_salary_5yr
) AS calc ON TRUE
ORDER BY calc.projected_salary_5yr DESC;

-- Example 14: LATERAL to reuse computed value multiple times
SELECT
    s.rep_name,
    s.amount,
    discounted.disc_price,
    discounted.disc_price * 0.1 AS tax_on_discounted
FROM sales s
JOIN LATERAL (
    SELECT
        CASE WHEN s.amount > 1500 THEN s.amount * 0.9
             WHEN s.amount > 1000 THEN s.amount * 0.95
             ELSE s.amount
        END AS disc_price
) AS discounted ON TRUE
ORDER BY s.amount DESC;
```

LEFT JOIN LATERAL

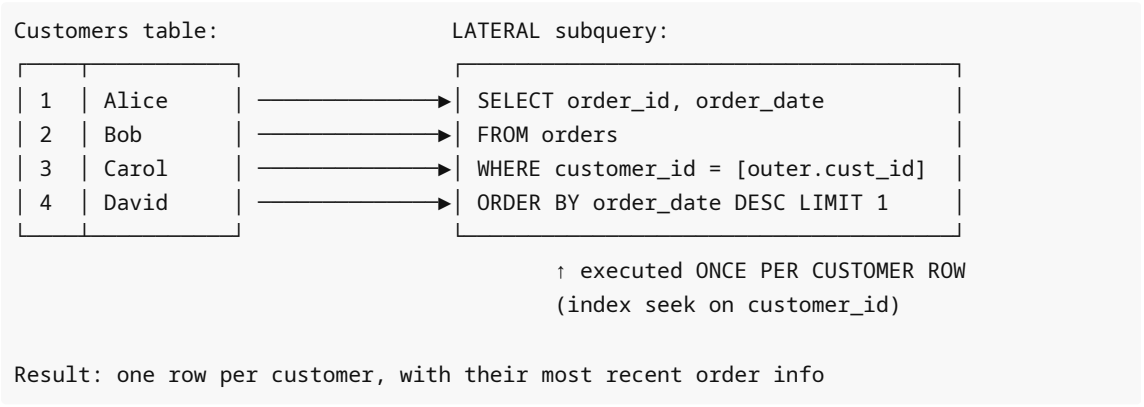
Use LEFT JOIN LATERAL when the subquery might return no rows (preserves outer rows).

```
-- Example 15: LEFT JOIN LATERAL — customers with or without orders
SELECT
    c.customer_name,
    COALESCE(lat.order_count, 0) AS order_count,
    COALESCE(lat.total_spent, 0) AS total_spent,
    lat.last_order_date
FROM customers c
LEFT JOIN LATERAL (
    SELECT
        COUNT(*)        AS order_count,
```

```
        SUM(total_amount) AS total_spent,
        MAX(order_date) AS last_order_date
    FROM orders o
    WHERE o.customer_id = c.customer_id
) AS lat ON TRUE
ORDER BY c.customer_name;
```

ASCII Visual Diagrams

LATERAL Execution Model



LATERAL vs Regular JOIN vs Correlated Subquery

Regular subquery in FROM:	LATERAL subquery:	Correlated subquery in SELECT:
Cannot see outer.col	CAN see outer.col	CAN see outer.col
Returns table result	Returns table result	Returns scalar ONLY
No LIMIT per outer row	LIMIT per outer row	No multi-row return
Executed once	Once per outer row	Once per outer row

Common Mistakes

Mistake 1: Using LATERAL without ON TRUE

```
-- WRONG: JOIN without ON condition for LATERAL
SELECT c.customer_name, lat.order_date
FROM customers c
JOIN LATERAL (SELECT order_date FROM orders WHERE customer_id = c.customer_id LIMIT
1) lat;
-- Missing ON condition

-- CORRECT:
... JOIN LATERAL (...) AS lat ON TRUE;
-- or implicit cross join:
... , LATERAL (...) AS lat
```

Mistake 2: Forgetting LEFT JOIN when no rows might exist

```
-- WRONG: INNER JOIN drops customers with no orders
FROM customers c
JOIN LATERAL (SELECT order_id FROM orders WHERE customer_id = c.customer_id LIMIT 1)
lat ON TRUE;

-- CORRECT: LEFT JOIN preserves customers even with no orders
FROM customers c
LEFT JOIN LATERAL (...) lat ON TRUE;
```

Mistake 3: Unnecessary LATERAL — use window function instead

```
-- LATERAL (when all rows are needed):
FROM emp_salary e JOIN LATERAL (SELECT AVG(salary) FROM emp_salary WHERE dept =
e.dept) lat ON TRUE
-- Better: window function (no per-row execution overhead):
AVG(salary) OVER (PARTITION BY department)
```

Best Practices

1. Use LATERAL when you need multiple columns OR multiple rows from the subquery.
2. Use correlated subquery in SELECT when only one scalar value is needed.
3. Use window functions (OVER) when you need aggregate values across the whole partition.
4. Use LEFT JOIN LATERAL to preserve rows when the subquery may return no rows.
5. Ensure an index exists on the LATERAL subquery's WHERE column for performance.
6. LATERAL + LIMIT is ideal for "latest N per group" — much cleaner than ROW_NUMBER.

Performance Considerations

```
-- LATERAL executes the subquery once per row of the outer table
-- This is O(N * subquery_cost) — same as correlated subquery

-- Key optimization: index on the lateral subquery's join column
-- For: WHERE o.customer_id = c.customer_id ORDER BY order_date DESC LIMIT 1
CREATE INDEX idx_orders_customer_date ON orders(customer_id, order_date DESC);
-- This makes each LATERAL execution an index scan + top-1 retrieval = O(log M)

-- Compare to ROW_NUMBER approach:
-- ROW_NUMBER scans all orders, assigns numbers, then filters
-- LATERAL + LIMIT uses an index seek + stops at row N

-- For small datasets: window function / ROW_NUMBER is often faster
-- For large datasets with a good index: LATERAL + LIMIT wins

EXPLAIN (ANALYZE, FORMAT TEXT)
SELECT c.customer_name, lat.order_date
```

```
FROM customers c
LEFT JOIN LATERAL (
    SELECT order_date FROM orders WHERE customer_id = c.customer_id
    ORDER BY order_date DESC LIMIT 1
) lat ON TRUE;
-- Look for: Nested Loop → Index Scan on orders using idx_orders_customer_date
```

Interview Questions & Answers

Q1: What does LATERAL do in a FROM clause?

A: LATERAL allows a subquery in the FROM clause to reference columns from tables that appear before it in the FROM list — effectively creating a correlated subquery in FROM. Without LATERAL, subqueries in FROM are independent and cannot reference outer table columns.

Q2: What is the difference between a correlated subquery in SELECT and a LATERAL join?

A: A correlated subquery in SELECT can only return a single value (scalar). A LATERAL join can return multiple rows and multiple columns. LATERAL also supports ORDER BY and LIMIT within the subquery, enabling "latest N per group" queries that are impossible with scalar correlated subqueries.

Q3: How do you solve "top-N per group" with LATERAL?

A: `FROM groups LEFT JOIN LATERAL (SELECT ... FROM details WHERE details.group_id = groups.id ORDER BY ... LIMIT N) AS top ON TRUE`. The LATERAL subquery runs once per group row, fetching only N rows with LIMIT.

Q4: When should you use LATERAL vs window functions?

A: Use window functions (OVER) when you need aggregated values computed over the whole partition without row filtering. Use LATERAL when you need multiple rows per outer row, or when you need LIMIT per group (window functions cannot LIMIT within a partition).

Q5: What is the performance implication of a LATERAL join?

A: LATERAL executes the subquery once per row of the preceding table. With a good index on the join predicate, each execution is fast (index seek). For large outer tables without an index, LATERAL can be slow. Window functions with OVER are typically more efficient for full-partition aggregations.

Q6: Can LATERAL be used without JOIN?

A: Yes. A comma-separated LATERAL in the FROM clause is implicit CROSS JOIN LATERAL. For example: `FROM table_a, LATERAL (subquery) AS alias`. Without LATERAL, commas would be a regular cross join.

Q7: Why use LEFT JOIN LATERAL instead of JOIN LATERAL?

A: LEFT JOIN LATERAL preserves rows from the outer table even when the LATERAL subquery returns no rows. This is equivalent to a LEFT JOIN — the right-side columns will be NULL when no rows match. Use it when the outer rows are meaningful even without matching data.

Q8: What is LATERAL useful for with functions?

A: Set-returning functions (SRFs) like UNNEST, generate_series, STRING_TO_TABLE, and jsonb_array_elements automatically use LATERAL semantics when placed in FROM. You can use them with LATERAL to expand arrays or sequences that depend on values from preceding tables.

Hands-on Exercises

Exercise 1: Latest Order per Customer

For each customer, find their most recent order (date and amount). Include customers with no orders (show NULL).

```
-- Solution:
SELECT
    c.customer_name,
    c.city,
    lat.order_date  AS last_order_date,
    lat.total_amount AS last_order_amount
FROM customers c
LEFT JOIN LATERAL (
    SELECT order_date, total_amount
    FROM orders o
    WHERE o.customer_id = c.customer_id
    ORDER BY order_date DESC
    LIMIT 1
) AS lat ON TRUE
ORDER BY c.customer_name;
```

Exercise 2: Top-2 per Region

For each sales region, find the top-2 reps by total revenue using LATERAL.

```
-- Solution:
SELECT r.region, t.rep_name, t.revenue, t.rnk
FROM (SELECT DISTINCT region FROM sales WHERE region IS NOT NULL) r
LEFT JOIN LATERAL (
    SELECT
        rep_name,
        SUM(amount) AS revenue,
        ROW_NUMBER() OVER (ORDER BY SUM(amount) DESC) AS rnk
    FROM sales s
    WHERE s.region = r.region
    GROUP BY rep_name
    ORDER BY revenue DESC
    LIMIT 2
) AS t ON TRUE
ORDER BY r.region, t.rnk;
```

Exercise 3: Discount Calculation with Reuse

Apply a discount to each sale amount based on a tier rule, then show the discounted price and a 10% tax on it. Use LATERAL to avoid repeating the CASE expression.

```
-- Solution:
SELECT
```

```
    rep_name,
    amount,
    d.discounted,
    ROUND(d.discounted * 0.10, 2) AS tax,
    ROUND(d.discounted * 1.10, 2) AS total_with_tax
FROM sales s
JOIN LATERAL (
    SELECT ROUND(s.amount * CASE
        WHEN s.amount > 2000 THEN 0.85
        WHEN s.amount > 1000 THEN 0.90
        ELSE 0.95
    END, 2) AS discounted
) AS d ON TRUE
ORDER BY amount DESC;
```

Exercise 4: Expanding a Series per Row

For each department in emp_salary, generate the next 3 quarter-end dates (from today).

```
-- Solution:
SELECT
    dept,
    gs.n,
    DATE_TRUNC('quarter', CURRENT_DATE + (gs.n || ' months')::INTERVAL)
        + INTERVAL '3 months - 1 day' AS quarter_end
FROM (SELECT DISTINCT department AS dept FROM emp_salary) d,
     LATERAL generate_series(3, 9, 3) AS gs(n)
ORDER BY dept, gs.n;
```

Advanced Notes

LATERAL with VALUES

```
-- Use LATERAL to inject inline computed constants per row
SELECT e.emp_name, e.salary, thresholds.bonus_rate, e.salary * thresholds.bonus_rate
AS bonus
FROM emp_salary e
JOIN LATERAL (
    SELECT CASE
        WHEN e.salary > 100000 THEN 0.20
        WHEN e.salary >  80000 THEN 0.15
        ELSE 0.10
    END AS bonus_rate
) AS thresholds ON TRUE;
```

LATERAL in UPDATE/DELETE (PostgreSQL)

```sql
-- LATERAL-like correlation is implicit in UPDATE subqueries
UPDATE employees e
SET salary = (
    SELECT AVG(salary) FROM employees WHERE department = e.department
) * 1.1
WHERE e.dept_rank = 1;
```

Cross-references

- **01_joins_complete.md** (03_Intermediate_SQL) — Regular JOIN types
- **03_subqueries.md** (03_Intermediate_SQL) — Correlated subqueries vs LATERAL
- **01_window_functions_intro.md** — Window functions as non-LATERAL alternative
- **02_ranking_functions.md** — ROW_NUMBER as alternative to LATERAL + LIMIT
- **08_interview_window_tasks.md** — LATERAL appears in advanced interview problems